

# Two Kinds of "Agentic AI"

The client brief and the POC guide describe two different projects that share a word. This sets out what each one actually does, which moves the needle for this business, how they connect, and the steps to build them across all three brands.

| DATE           | APPLIES TO         | DOCUMENTS                |
|----------------|--------------------|--------------------------|
| 11 August 2026 | Solana · BBQ · OKO | Client brief + POC guide |

## 1. The core distinction

The two documents point in **opposite directions**. This is the single most important thing to get straight before any budget is committed.

|                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>A — Client brief: "Agentic AI Friendly"</b></p> <p><b>Inbound.</b> Making our sites readable by <i>other people's</i> agents — ChatGPT, Claude, Perplexity, shopping assistants — so we appear when someone asks them for a grill.</p> <p>We build data. Someone else's AI consumes it. No AI runs on our infrastructure.</p> | <p><b>B — POC guide: "Agentic AI Development"</b></p> <p><b>Inward.</b> Building <i>our own</i> AI shopping assistant — a chat box on our site, powered by a local Llama 3.2 model calling our Searchkit and product APIs.</p> <p>We run the AI. It serves customers already on our site. A product feature, not a visibility measure.</p> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|               | A — AGENT-FRIENDLY SITE                                                      | B — ON-SITE AI AGENT                                       |
|---------------|------------------------------------------------------------------------------|------------------------------------------------------------|
| Who it serves | Shoppers using an external AI who have never heard of us                     | Shoppers already on our website                            |
| Wins us       | Discovery. Being on the shortlist.                                           | Conversion. Better on-site help.                           |
| Deliverable   | llms.txt, structured data, server-rendered pages, crawler policy, public API | Chat UI, agent runtime, tool layer, evaluation suite       |
| Runs AI?      | No                                                                           | Yes — model inference per conversation                     |
| Ongoing cost  | None                                                                         | Inference, hosting, monitoring, tuning                     |
| Risk          | Very low — standard web practice                                             | Moderate — hallucination, prompt injection, latency, spend |
| Also helps    | Ordinary Google SEO — same work                                              | Nothing else                                               |
| If skipped    | Invisible to AI shopping entirely                                            | Site works exactly as it does today                        |

**They are not alternatives, and B does not deliver A.** A chat box on our website does nothing to make us appear when a shopper asks ChatGPT for a grill — the assistant never visits our chat box. If the client's actual goal is the one stated in their brief ("a growing share of discovery and comparison is driven by autonomous AI agents"), then **A is the project that addresses it and B does not**.

## 2. Which one actually impacts this app

**A — decisively — and it is not close.** Three reasons:

- It is a prerequisite.** If external agents cannot read our catalogue, no on-site chatbot changes that. A is foundational; B is optional polish on top.
- It pays for itself twice.** Server-rendered products and structured data are ordinary technical SEO. They improve normal Google results whether or not a single AI agent ever visits. B has no such second use.
- The risk profile is not comparable.** A is well-established web practice with essentially no downside. B puts a language model in front of customers, where a hallucinated price or stock claim is a commercial problem — and a product description is untrusted text an attacker can write.

**Where A already stands.** Most of it is built and verified across all three brands, on the `agentic-ai-wip` branch. Listing pages went from **0 to 30 products** readable without JavaScript; the storefront previously rendered a blank page to anything that does not run JS. Structured data, crawler policy, `llms.txt`, `llms-full.txt`, semantic markup and API rate limiting are all in place. Roughly **65%** of the client brief is complete.

### 3. Where the two projects genuinely meet

There is one real overlap, and it is worth designing around rather than building twice.

The POC needs three tools: `searchProducts()` , `getProduct()` , `getInventory()` . **Those are the same three endpoints the client brief asks for in its section 3** ("expose public, rate-limited read endpoints for product catalogs, search, and inventory checks").



Build it once and both documents are served. Build the POC first in isolation, and the tools end up private, undocumented, and needing to be re-exposed later — the same work done twice.

**One gap to flag now:** the POC assumes a `getInventory()` backed by live stock. We do not really have that. The catalogue carries a published flag and variant data, not a live inventory feed. Before promising an agent that answers "is this in stock", someone needs to confirm where authoritative stock actually lives.

### 4. Honest assessment of the POC guide

| POINT                                    | ASSESSMENT                                                                                                                                                                     |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reuse existing Searchkit / Elasticsearch | <b>Sound</b> We already have this; the search endpoint exists and is now rate-limited.                                                                                         |
| Provider-neutral wrapper                 | <b>Sound</b> Genuinely good advice. Swapping the model later should not touch the commerce tools.                                                                              |
| Read-only tools; no checkout             | <b>Sound</b> Correct scope for a first build.                                                                                                                                  |
| Prompt-injection test                    | <b>Sound</b> Treating product copy as untrusted data is exactly right.                                                                                                         |
| 50–100 evaluation questions              | <b>Sound</b> The difference between a demo and something shippable.                                                                                                            |
| Llama 3.2 3B                             | <b>Caution</b> A 3B model is weak at reliable multi-step tool calling over a 6,000-product catalogue. Fine to prove the wiring; expect a larger model before customers see it. |
| "Free AI"                                | <b>Caution</b> True only for local development. Production inference is a real recurring bill, as the guide itself notes.                                                      |
| Local Ollama                             | <b>Caution</b> Runs on a developer machine. Not a deployment path — production needs hosted inference.                                                                         |
| Example catalogue                        | <b>Note</b> The worked example is shipping containers, not grills. The guide is a generic template from another project; the architecture transfers, the specifics do not.     |

### 5. Recommendation

**Finish A. Then build the tool layer. Then, if the client still wants it, do B on top.** That order costs the least, de-risks the most, and means nothing is built twice.

| ORDER | WHAT                        | WHY THIS POSITION                                                                                                            |
|-------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 1     | Finish the client brief (A) | Already ~65% done. Cheap, low risk, helps SEO regardless. Nothing else should jump ahead of it.                              |
| 2     | Build the shared tool layer | Satisfies brief §3 <i>and</i> becomes the POC's tools. The single highest-leverage piece of work available.                  |
| 3     | Publish OpenAPI + MCP       | Makes the tool layer usable by external assistants. Small once step 2 exists, and the genuinely differentiating deliverable. |
| 4     | POC agent (B)               | Now cheap — the tools already exist and are tested. Scope it as a demo, quote production separately.                         |

## 6. Implementation — step by step

All three brands share one codebase, so every item below lands on Solana, BBQ and OKO at once. Brand differences come from `STORE_ID`, exactly as the storefront already works. Each brand is a separate deployment and all three must be released.

### Phase 1 — Finish the client brief

READY NOW · MOSTLY BLOCKED ON CLIENT ANSWERS, NOT ENGINEERING

| ID  | TASK                                           | NOTES                                                                        |
|-----|------------------------------------------------|------------------------------------------------------------------------------|
| A-1 | Obtain published shipping & returns terms      | Client. The one structured-data field we cannot fill; guessing is penalised. |
| A-2 | Decide AI training-crawler policy              | Client. All 22 currently allowed. One-line change.                           |
| A-3 | Clear BBQ & OKO social env vars                | Both still publish Solana's accounts, asserting the brands are one entity.   |
| A-4 | Confirm hosting bot-protection settings        | Can silently block AI crawlers and undo the robots.txt work.                 |
| A-5 | Publish shipping/returns in structured data    | Unblocked by A-1.                                                            |
| A-6 | Audit the product feed                         | Confirm GTIN, stock and shipping populated.                                  |
| A-7 | Merge <code>agentic-ai-wip</code> back to main | Includes the BBQ crash fix; verify all three brands build.                   |

### Phase 2 — The shared tool layer

THE PIVOT POINT · SERVES BOTH PROJECTS

| ID  | TASK                                      | NOTES                                                                                      |
|-----|-------------------------------------------|--------------------------------------------------------------------------------------------|
| B-1 | Confirm where authoritative stock lives   | Blocks B-4. We do not currently have a live inventory source.                              |
| B-2 | <code>searchProducts</code> endpoint      | Thin, validated wrapper over the existing Searchkit layer. Never pass raw queries through. |
| B-3 | <code>getProduct</code> endpoint          | Single product by handle or SKU. Return only needed fields.                                |
| B-4 | <code>getInventory</code> endpoint        | Live read-through, bypassing page caches so stock is never stale.                          |
| B-5 | Brand scoping via <code>STORE_ID</code>   | Each deployment answers only for its own catalogue.                                        |
| B-6 | Extend rate limiting to the new endpoints | Mechanism already built; reuse it. Must ship together, not after.                          |
| B-7 | Strict input validation & error schema    | Typed JSON errors so an agent can react rather than guess.                                 |

### Phase 3 — Make the tools discoverable

SMALL ONCE PHASE 2 EXISTS · COMPLETES THE CLIENT BRIEF

| ID  | TASK                             | NOTES                                                                                      |
|-----|----------------------------------|--------------------------------------------------------------------------------------------|
| C-1 | Publish an OpenAPI specification | At the conventional path, referenced from <code>llms.txt</code> and the API root.          |
| C-2 | Crawler-permission carve-out     | <code>/api/*</code> is currently disallowed wholesale, which would hide the new endpoints. |
| C-3 | MCP server over the same tools   | Lets assistants query the catalogue natively. The differentiating deliverable.             |
| C-4 | Verify with real assistants      | End-to-end check that an external agent can actually answer a product question.            |

### Phase 4 — The on-site agent (the POC)

ONLY AFTER PHASES 2–3 · SCOPE AS A DEMO, QUOTE PRODUCTION SEPARATELY

| ID  | TASK                                    | NOTES                                                                                       |
|-----|-----------------------------------------|---------------------------------------------------------------------------------------------|
| D-1 | Provider-neutral AI wrapper             | Per the guide. Local model in development, hosted later, commerce tools untouched.          |
| D-2 | Agent endpoint with a tool-calling loop | Server keeps control of every API call; hard step and timeout limits.                       |
| D-3 | Bind the Phase 2 tools                  | Already built, already validated, already rate-limited.                                     |
| D-4 | Brand-aware system instructions         | Persona and catalogue scope from <code>STORE_ID</code> so one build serves three stores.    |
| D-5 | Chat interface                          | Themed per brand like every other component.                                                |
| D-6 | Evaluation suite, 50–100 questions      | Per the guide. Includes failure and injection cases. The line between demo and shippable.   |
| D-7 | Prompt-injection hardening              | Product copy is attacker-writable. Price and stock must come from the API, never the model. |
| D-8 | Decide the production model & host      | A 3B local model will not carry this in production. Real recurring cost.                    |

## 7. What to tell the client

- **The brief and the POC are different projects.** Worth confirming which outcome they actually want before scoping either.
- **The brief is ~65% delivered** and most of what remains is waiting on their answers, not on engineering.
- **The POC does not deliver the brief.** If the goal is showing up in AI shopping answers, an on-site chat box does not do that.
- **Build the tool layer once.** It is the only place the two projects genuinely overlap, and doing it first makes the POC substantially cheaper.
- **"Free AI" means no per-token bill during local development.** Production inference, hosting and maintenance are separate and recurring.

**Basis.** Figures measured on local development builds of all three brands, 7–11 August 2026. Engineering detail is maintained alongside the code at `docs/agent-ai-readiness.md` ; the completed work sits on the `agent-ai-wip` branch. Model and protocol specifics in the POC guide should be re-checked before committing budget — that part of the landscape moves quickly.